

How Computers Work · Section 5

What an Operating System Does

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. The OS as the Manager 1

2. The Kernel 2

3. User Space and Kernel Space 2

4. System Calls 3

5. Drivers and Hardware 3

6. Multitasking 4

7. Users, Permissions, and the OS as a Security Boundary 4

Common Mistakes 5

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- What an operating system is, and why every computer needs one.
- The kernel: the core of the OS.
- The split between user space and kernel space.
- System calls: how programs ask the OS for things.
- Drivers, and how the OS talks to hardware.
- Multitasking: how one computer runs many programs.
- Users and permissions, and why the OS is the main security boundary.

Before You Start

Read lessons 1 to 4. You know the hardware: the CPU, memory, and storage. This lesson is about the software that manages all of it, the operating system. No code in this lesson; it builds the mental model that the Linux (03) and Windows (05) courses, and most of the defensive courses, assume.

---

1. The OS as the Manager

Between the bare hardware and the apps you use sits the **operating system** (OS): Windows, macOS, Linux, Android, iOS. From lesson 1's three layers, the OS is the middle one. Its job, in one sentence: **the operating system manages the computer's hardware and resources, and shares them safely among all the programs that want to use them.**

Think of a busy restaurant kitchen. The hardware is the stoves, the fridge, the knives. The programs are the cooks, all wanting to use the same equipment at once. Without a manager, they would collide, grab each other's ingredients, and set the place on fire. The OS is the kitchen manager: it decides who uses what, when, and keeps them out of each other's way.

Concretely, the OS:

- **Shares the CPU** among programs (multitasking, topic 6).
- **Manages memory**, giving each program its own space and keeping them apart.
- **Controls access to storage**, the network, the screen, and other hardware.
- **Enforces the rules** about who is allowed to do what (topic 7).

Every program runs *on top of* the OS and asks it for everything it needs. A program cannot touch the disk or the network directly; it asks the OS. That funnel, every request passing through the OS, is what makes the OS the center of a computer's security.

2. The Kernel

At the heart of the operating system is the **kernel**: the core program that is always running and has complete control over the machine. If the OS is the manager, the kernel is the manager's brain and hands. It is the part that actually talks to the hardware, shares out the CPU and memory, and enforces the rules.

The kernel is special because it runs with **full privilege**: it can do anything to the hardware and to any program. Everything else, your apps, even most of the OS's own tools, runs with less power and must go through the kernel for anything important. This is deliberate. Because the kernel can do anything, it is kept small, carefully written, and protected, and everything else is kept at arm's length from the hardware.

For security, the kernel is the ultimate prize. **If an attacker takes control of the kernel, they control the entire machine**, every program, every file, every secret, with nothing above them to say no. This is why the most serious attacks (and the most powerful malware, like **rootkits**, which you will study in Malware Analysis) aim to reach the kernel, and why defending the kernel's integrity matters so much.

3. User Space and Kernel Space

To protect the kernel, the CPU itself enforces a hard line between two worlds:

- **Kernel space:** where the kernel runs, with full privilege and direct access to all hardware and memory.
- **User space:** where normal programs run, your browser, your editor, with limited privilege and no direct access to hardware.

This is sometimes called the difference between **kernel mode** and **user mode**, and it is enforced by the processor, not just by convention, which makes it a strong boundary. A program in user space simply *cannot* reach out and touch the disk, the network card, or another program's memory directly. The hardware forbids it.

Why build this wall? **Isolation and safety.** If a user-space program crashes or misbehaves, it cannot take down the whole machine or corrupt the kernel, because it never had the power to. A buggy app harms only itself. A malicious app is boxed in. The wall between user space and

kernel space is one of the most important security features in any computer, and crossing it, getting code to run in kernel space, is a major goal in both attack (privilege escalation, course 13) and defense.

4. System Calls

If user-space programs cannot touch hardware directly, how does your editor save a file to disk? It **asks the kernel**, through a **system call** (often shortened to **syscall**).

A system call is the official, controlled doorway from user space into kernel space. When a program needs something only the kernel can do, open a file, read from the network, start another program, allocate memory, it makes a system call: it asks the kernel, "please do this for me." The kernel checks whether the program is allowed, does the work if so, and hands back the result. The program never touches the hardware itself; the kernel does it on the program's behalf.

System calls are the complete list of everything a program can ask the computer to do. That makes them deeply important for security:

- **They are the checkpoints.** Every file opened, every network connection, every new program, goes through a system call, where the kernel can enforce permissions. This is where the OS says yes or no.
- **They are how we watch programs.** Security tools and investigators observe a program's system calls to see what it *really* does, what files it touches, what it sends on the network, regardless of what it claims. Malware analysis (course 29) leans heavily on this: you learn far more from the calls a program makes than from what it says.

So the system-call boundary is both the OS's main control point and a prime place to observe behavior. Keep the picture: a program wanting anything real must knock on the kernel's door with a system call, and the kernel decides.

5. Drivers and Hardware

Computers have endless different hardware: thousands of models of printers, graphics cards, Wi-Fi adapters, webcams. The kernel cannot possibly know the details of every one. So it uses **drivers**: pieces of software, usually part of or loaded into the kernel, that each know how to talk to one specific kind of hardware.

A driver is a translator. The kernel speaks to a driver in general terms ("send this data to the printer"), and the driver turns that into the exact signals that particular printer understands. This is why you sometimes "install drivers" for new hardware: you are giving the kernel the

translator it needs for that device.

Drivers carry a special security weight, because **most drivers run in kernel space**, with full privilege. That makes them a favorite target and tool for attackers: a flaw in a driver is a flaw *in the kernel's world*, often leading straight to full control of the machine. Malicious or vulnerable drivers are a real and serious attack path (studied in exploitation and Windows security). The lesson for now: drivers are powerful, privileged code, and powerful privileged code is always a security concern.

6. Multitasking

Your computer seems to run many programs at once, a browser, music, a chat app, even though, as lesson 4 showed, each CPU core can really only execute one stream of instructions at a time. How? The OS performs **multitasking**: it switches the CPU between programs so quickly that they all appear to run at the same time.

The kernel gives each program a tiny slice of CPU time, a few milliseconds, then pauses it, saves its state (its registers, its place in its instructions), and switches to the next program. This swap is called a **context switch**, and it happens thousands of times a second. With several cores (lesson 4), the computer genuinely runs several programs at once *and* switches among many more on each core. The result is the smooth illusion that everything runs together.

The unit the OS switches between is the **process**, a running program, which is the entire subject of the next lesson. For now, the key security points:

- **The OS decides who runs,** and can stop, pause, or kill any program. That control is a defensive tool (ending a malicious process) and a target (malware tries to keep itself running).
- **Programs are kept apart during switching.** Each process's state is saved and isolated, so one program cannot read another's working data just because they share the CPU. That isolation is, again, a core security guarantee the OS provides.

7. Users, Permissions, and the OS as a Security Boundary

We end where the OS matters most for security. The operating system does not just manage hardware; it enforces **who is allowed to do what.** This builds directly on Security Fundamentals: authentication (who you are) and authorization (what you may do) are enforced, on a real computer, largely by the OS.

Every modern OS has the idea of **users** (accounts), and every file and action has **permissions** that say which users may do it. There is usually a specially powerful account, called **root** (or the **superuser**) on Linux and macOS, and **Administrator** on Windows, which can do almost anything, change the system, read any file, install software. Ordinary user accounts are deliberately limited.

This is the OS applying least privilege (Security Fundamentals, lesson 7) in practice:

- A normal user can run their own programs and edit their own files, but not change the system or read other users' private data.
- Only the powerful account can make system-wide changes, which is why malware and attackers work so hard to **escalate privileges**, to go from an ordinary account to root or Administrator. That jump is the single most common goal after breaking in, and a whole part of the pentesting courses (13) is about it.

So the OS is **the main security boundary on a computer.** It isolates programs from each other (user space vs kernel space), controls every real action (system calls), and enforces who may do what (users and permissions). Nearly every attack is, in some form, an attempt to break one of these OS-enforced boundaries, and nearly every defense is about keeping them intact. That is why the OS gets its own deep courses (Linux and Windows) and why understanding it here is essential.

You now understand the hardware (lesson 4) and the OS that manages it (this lesson). Next, we zoom into the single most important thing the OS manages, and the place where programs actually live and run: processes and their memory.

Common Mistakes

1. Thinking apps talk to hardware directly. They do not. User-space programs ask the kernel, through system calls, for anything real. The kernel is the gatekeeper.

2. Underestimating the kernel. Control the kernel and you control the whole machine. That is why it is small, protected, and the top prize for attackers.

3. Ignoring the user/kernel space boundary. It is enforced by the CPU itself, and crossing it is a major attack goal (privilege escalation) and defense concern.

4. Treating drivers as harmless. Most run in kernel space with full privilege, so a flawed driver is a flaw in the kernel's world, a serious attack path.

5. Forgetting that privilege escalation is the usual next step. After getting in as an ordinary user, attackers try to become root/Administrator. Guarding that jump is central to defense.

Summary

Term	Remember
Operating system	Manages hardware and shares it safely among programs; the middle layer
Kernel	The core, always-running, fully privileged part; control it and you control the machine
User vs kernel space	Limited programs vs the privileged kernel; a hard, CPU-enforced boundary
System call	The controlled doorway programs use to ask the kernel for anything real; a checkpoint and a watch point
Driver	Privileged software that lets the kernel talk to specific hardware; a serious attack surface
Multitasking	The OS switches the CPU among programs fast (context switches), keeping them apart
Users & permissions	The OS enforces who may do what; root/Administrator is all-powerful; attackers escalate to it

Check Your Understanding

1. In one sentence, what does an operating system do?
2. What is the kernel, and why is it the top prize for an attacker?
3. What is the difference between user space and kernel space, and what enforces it?
4. How does a normal program get the OS to save a file or use the network?
5. Why are drivers a special security concern?
6. What is a context switch, and what illusion does it create?
7. What is privilege escalation, and why do attackers want it?

Answers

1. It manages the computer's hardware and resources and shares them safely among all the programs that want to use them.
2. The core program that is always running with full control of the machine. Controlling it means controlling everything, every program, file, and secret, so it is the top target.
3. User space is where limited, normal programs run with no direct hardware access; kernel space is where the fully privileged kernel runs. The CPU itself enforces the boundary.

4. It makes a system call: it asks the kernel to do it. The kernel checks permissions, does the work, and returns the result.
5. Most drivers run in kernel space with full privilege, so a flaw in a driver is a flaw in the kernel's world, often leading to full control of the machine.
6. The OS pausing one program, saving its state, and switching the CPU to another. Done thousands of times a second, it creates the illusion that many programs run at once.
7. Going from an ordinary, limited account to a powerful one (root/Administrator). Attackers want it because the powerful account can change the system and read any data, so it is the usual goal after breaking in.

Practice

1. Explain, using the kitchen-manager analogy, why programs are not allowed to touch the hardware directly, and what they do instead.
2. Name one action your own computer does that must involve a **system call** (for example, saving a file), and say which part, user space or kernel space, actually carries it out.
3. On your computer, you have a normal user account and there is a powerful admin/root account. Give one thing the admin account can do that your normal account cannot, and explain why that limit is a security feature, not just an annoyance.

A Worked Example

1. In the kitchen, cooks (programs) don't each grab the single oven whenever they like, or they'd collide and cause chaos. They ask the manager (the kernel), who decides who gets it and when. Programs likewise ask the kernel, via system calls, instead of touching hardware directly, so the kernel can keep order and enforce the rules.
2. Saving a document must involve a system call to write to the disk. User space (the app) asks; kernel space (the kernel) actually writes the bytes to storage.
3. The admin account can install system-wide software or change security settings; my normal account cannot. That limit is a security feature because if malware runs under my limited account, it also cannot change the system, so the damage is contained, exactly the least-privilege idea from Security Fundamentals.

Course mark icon from Material Symbols by Google, Apache License 2.0.